

GeoPeople

Rapport global de développement

Adam Anthonin Ihssan

Juin 2026

Avertissement : nos rendus personnels sont dans le dossier nos_rendu_perso.

Avertissement : le dépôt officiel du projet est <https://github.com/adxdits/GeoPeople>.

Table des matières

1	Introduction	3
2	Objectifs fonctionnels du projet	3
3	Architecture globale	4
3.1	Vue d'ensemble	4
3.2	Composants principaux côté Android	4
3.3	Composants principaux côté backend	4
3.4	Diagramme UML simplifié (coloré)	5
4	Implémentation des mécanismes de jeu	5
4.1	Découverte et placement des cartes	5
4.2	Acquisition d'une carte	5
4.3	Inventaire, collections et score	5
4.4	Echanges de cartes	6
5	Mini-jeux et capteurs	6
6	Biographies et données externes	6
7	Qualité logicielle et tests	6
7.1	Modularité	6
7.2	Tests existants	7
7.3	Points de robustesse déjà présents	7
8	Répartition du travail	7
9	Organisation d'équipe et gestion des conflits	7
9.1	Organisation mise en place	7
9.2	Exemples de conflits techniques rencontrés et résolus	8
9.3	Gestion des risques projet	8

10 Limites actuelles et pistes d'amélioration	8
11 Conclusion	8
A Annexe A – Endpoints principaux	8

1 Introduction

GeoPeople est une application Android de jeu géolocalisé inspirée du principe de *capture en mobilité*. Le joueur se déplace physiquement, détecte des cartes biographiques autour de sa position, réussit un mini-jeu, puis capture la carte pour enrichir son inventaire et améliorer son score.



FIGURE 1 – Illustration principale de GeoPeople (image utilisée dans le README).

Le projet repose sur deux blocs techniques :

- un frontend Android en Kotlin/Jetpack Compose ;
- un backend Node.js/TypeScript (Express) exposant les règles métier (capture, score, échanges, classement).

Ce document présente l'architecture, les choix techniques, l'organisation d'équipe, les difficultés rencontrées et les solutions apportées.

2 Objectifs fonctionnels du projet

Les objectifs implémentés dans l'état actuel du projet sont les suivants :

- suivi GPS du joueur ;
- affichage cartographique (OpenStreetMap via osmdroid) ;
- chargement de cartes proches ;
- tentative de capture conditionnée à la distance et à la réussite d'un mini-jeu ;
- verrouillage temporaire en cas d'échec du mini-jeu ;
- inventaire des cartes capturées ;

- calcul de score avec multiplicateurs par collections ;
- classement global des joueurs ;
- proposition/acceptation/rejet d'échanges de cartes entre joueurs proches ;
- historique de possession des cartes (capture et échange) ;
- consultation de biographies via Wikidata et Wikipedia.

3 Architecture globale

3.1 Vue d'ensemble

L'application suit une architecture client-serveur :

- le frontend gère l'expérience de jeu, la géolocalisation, la carte et l'interface ;
- le backend centralise les règles de validation pour garantir une cohérence multi-joueurs.

Le backend expose des routes REST sous `/api` :

- `/api/cards`
- `/api/players`
- `/api/captures`
- `/api/trades`

3.2 Composants principaux côté Android

- **MainActivity** : point d'entrée, gestion de la permission de localisation.
- **GameViewModel** : orchestration principale (GPS, synchro backend, capture, score, échanges).
- **LocationService** : récupération des positions via **FusedLocationProviderClient**.
- **CardRepository** : stockage des cartes courantes et fallback Wikidata.
- **ApiService** : client HTTP (OkHttp) vers le backend.
- **UI Compose** : navigation, carte, inventaire, profil, leaderboard, détails carte.

3.3 Composants principaux côté backend

- **cardsService** : liste de cartes, filtrage géographique, génération de cartes de démonstration autour du joueur.
- **captureService** : validation de capture (distance, mini-jeu, verrouillage progressif).
- **playerService** : inscription, inventaire, mise à jour de position, anti-triche basique, classement.
- **scoringService** : calcul du score total et des collections avec multiplicateurs Fibonacci.
- **tradeService** : cycle de vie des propositions d'échange et contrôle de proximité.

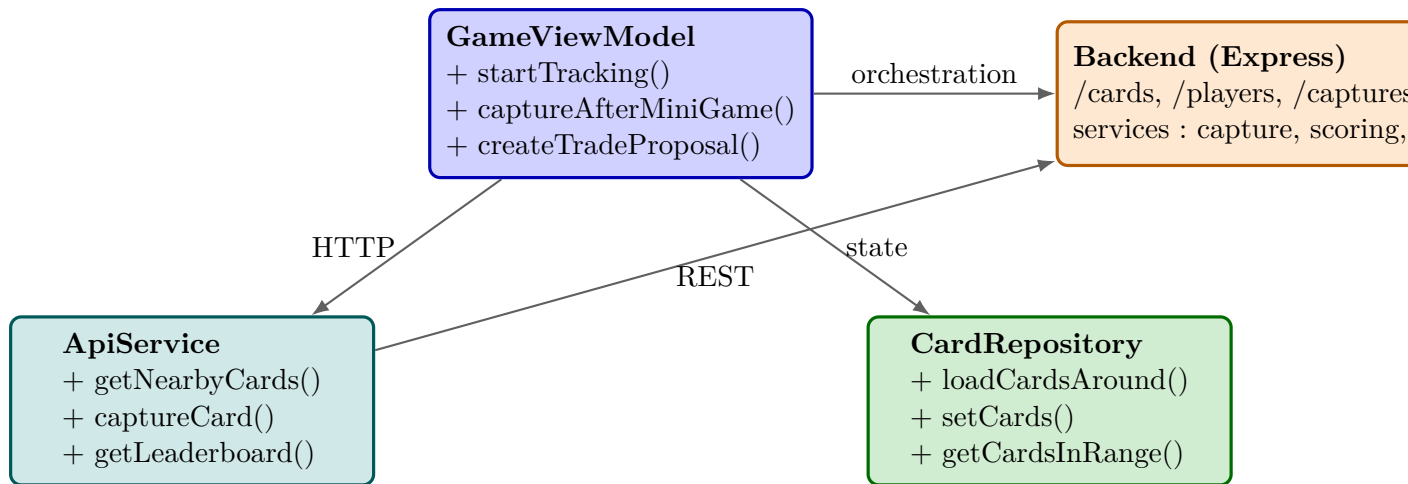


FIGURE 2 – UML simplifié des composants principaux de GeoPeople.

3.4 Diagramme UML simplifié (coloré)

4 Implémentation des mécanismes de jeu

4.1 Découverte et placement des cartes

Le frontend demande périodiquement des cartes proches de la position du joueur. Le backend fournit une liste filtrée par rayon (`/api/cards/nearby?lat=...&lon=...&radius=...`).

Particularité utile en démonstration : le backend peut générer dynamiquement des cartes *demo* autour des coordonnées du joueur afin d'assurer une jouabilité locale.

4.2 Acquisition d'une carte

Le flux réel implémenté est :

1. le joueur sélectionne une carte visible ;
2. un mini-jeu est lancé aléatoirement parmi les mini-jeux disponibles ;
3. si réussite, appel backend de capture avec `miniGameSuccess=true` ;
4. le backend vérifie notamment :
 - que la carte existe et n'est pas déjà capturée ;
 - que le joueur est à moins de 50 m ;
 - que la carte n'est pas verrouillée pour ce joueur.
5. en cas d'échec du mini-jeu, verrouillage progressif (30 s, 2 min, 5 min, 15 min, 30 min).

4.3 Inventaire, collections et score

Le score ne dépend pas seulement de la puissance brute des cartes. Le backend regroupe les cartes en collections selon plusieurs critères implémentés :

- initiales du nom de la personnalité ;
- lieu ;
- relation (naissance, décès, etc.) ;
- première lettre du nom.

Pour chaque collection de taille $n \geq 2$, les coefficients appliqués suivent Fibonacci :

$$F_1 = 1, \quad F_2 = 1, \quad F_n = F_{n-1} + F_{n-2}.$$

Si une carte appartient à plusieurs collections, les coefficients sont additionnés, puis appliqués à sa puissance. Ce choix récompense les inventaires variés et cohérents.

4.4 Echanges de cartes

Un échange n'est possible que si :

- les deux joueurs existent ;
- chacun possède la carte proposée ;
- les joueurs sont physiquement proches (seuil backend : 100 m) ;
- la proposition est encore **pending**.

Le backend gère ensuite l'acceptation/refus et met à jour l'historique de possession des cartes en cas d'échange validé.

5 Mini-jeux et capteurs

Le projet intègre plusieurs mini-jeux, dont au moins un par membre :

- **Anthonin** : mini-jeu type Snake contrôlé par l'accéléromètre (inclinaison du téléphone).
- **Ihssan** : mini-jeu *Ball Run/Treasure* utilisant les données de mouvement (accéléromètre) pour piloter la progression.
- **Adam** : mini-jeu *Pokeball* combinant accéléromètre et détection micro (clap detector) ; mini-jeu *Memory* intégré comme mode complémentaire.

Dans l'application principale, le mini-jeu de capture est sélectionné aléatoirement parmi les activités disponibles.

6 Biographies et données externes

Deux sources externes sont exploitées :

- **Wikidata** pour les entités, lieux et métadonnées ;
- **Wikipedia** (API `/page/summary`) pour le résumé et certaines images.

Le frontend fusionne ces informations dans un modèle **BiographyDetails**, puis les affiche dans une interface dédiée sans composant WebView HTML.

7 Qualité logicielle et tests

7.1 Modularité

La séparation en services backend, ViewModel Android, repository, et composants UI Compose rend l'application maintenable et extensible.

7.2 Tests existants

Le backend inclut une suite de tests ciblant le service de score (cas sans collection, Fibonacci, cumul de multiplicateurs). Ces tests valident la logique de calcul des points, qui est centrale pour la compétition.

7.3 Points de robustesse déjà présents

- validation stricte des payloads API ;
- messages d'erreur contextualisés ;
- fallback frontend en cas d'indisponibilité backend ;
- vérifications anti-abus sur capture et échange.

8 Répartition du travail

La répartition consolidée du projet est :

Membre	Part estimée	Contributions principales
Adam	40%	Intégration générale Android, mini-jeux (Pokeball/Memory), capture flow, racc
Anthonin	30%	Mini-jeu Snake (capteurs), logique de jeu, stabilisation UX de captu
Ihssan	30%	Mini-jeu Ball Run/Treasure (capteurs), écrans de jeu, support intégration

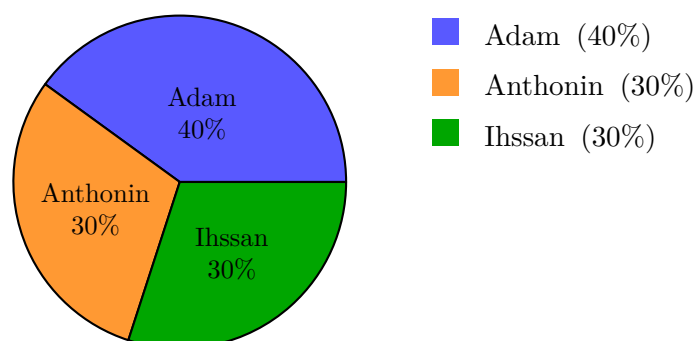


FIGURE 3 – Cercle de répartition du travail du projet.

9 Organisation d'équipe et gestion des conflits

9.1 Organisation mise en place

Nous avons structuré le travail avec une logique simple :

- lots par domaine (backend, UI/navigation, mini-jeux) ;
- points de synchronisation hebdomadaires ;
- revues croisées avant fusion dans la branche principale ;
- arbitrage technique sur la base de prototypes rapides.

9.2 Exemples de conflits techniques rencontrés et résolus

1. **Conflit de priorités (jouabilité vs. rigueur métier)** : une partie de l'équipe souhaitait une capture très permissive pour fluidifier les tests, l'autre insistait sur des contraintes strictes (distance, verrouillage, unicité).
Résolution : règles strictes conservées côté backend, avec données de démonstration proches du joueur pour garder une bonne jouabilité.
2. **Conflit d'intégration des mini-jeux** : des écarts de structure (naming package/activité) ont ralenti le branchement dans l'appli principale.
Résolution : définition d'un contrat d'intégration unique (résultat `Activity.RESULT_OK` pour succès), puis ajout d'un lanceur commun dans l'écran de capture.
3. **Conflit de stratégie score** : débat entre score linéaire simple et système de collections plus riche.
Résolution : adoption des multiplicateurs Fibonacci avec tests unitaires pour fiabiliser la formule.

9.3 Gestion des risques projet

Nous avons adopté une approche pragmatique :

- risque backend indisponible : message utilisateur + fallback local ;
- risque régressions score : tests automatisés ;
- risque incohérences multi-joueurs : validations serveur systématiques ;
- risque dérive planning : gel fonctionnel progressif en fin de sprint.

10 Limites actuelles et pistes d'amélioration

- persistance backend actuellement en mémoire (pas de base de données durable) ;
- anti-triche encore minimal (améliorable avec règles serveur temporelles enrichies) ;
- couverture de tests à élargir (captures, échanges, routes API) ;
- amélioration possible du mode *découvrable* et des interactions joueurs temps réel.

11 Conclusion

GeoPeople constitue une base fonctionnelle solide d'application géolocalisée multi-mécanismes : carte, capture conditionnée à mini-jeu, inventaire, score avancé et échanges. Le projet démontre une architecture claire, des règles métier cohérentes et une bonne répartition du travail.

Les prochaines itérations visent surtout la persistance durable, le renforcement anti-triche, et l'approfondissement de l'expérience multijoueur temps réel.

A Annexe A – Endpoints principaux

Route	Description
GET /api/health	Etat du backend

GET	Cartes proches d'une position
/api/cards/nearby	
GET	Historique de possession d'une carte
/api/cards/:id/history	
POST	Inscription/récupération d'un joueur
/api/players/register	
PUT	Mise à jour position joueur
/api/players/:id/location	
GET	Classement global
/api/players/leaderboard	
GET	Inventaire détaillé d'un joueur
/api/players/:id/inventory/cards	
POST /api/captures	Tentative de capture (succès/échec mini-jeu)
POST /api/trades	Création proposition d'échange
PUT	Acceptation d'échange
/api/trades/:id/accept	
PUT	Refus d'échange
/api/trades/:id/reject	
